

**Image Processing**  
**Unit 2**  
**Image Enhancement and Filtering in the**  
**Spatial Domain**

Mathematical Examples with Solutions



Prepared by: Kiran Bagale

November, 2025

# Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Basic Gray Level Transformations</b>    | <b>2</b>  |
| 1.1      | Contrast Stretching . . . . .              | 2         |
| 1.2      | Digital Negative . . . . .                 | 3         |
| 1.3      | Thresholding . . . . .                     | 4         |
| 1.4      | Log Transformation . . . . .               | 5         |
| 1.5      | Power-Law (Gamma) Transformation . . . . . | 6         |
| <b>2</b> | <b>Histogram Processing</b>                | <b>8</b>  |
| 2.1      | Histogram Equalization . . . . .           | 8         |
| 2.2      | Histogram Statistics . . . . .             | 10        |
| <b>3</b> | <b>Spatial Filtering</b>                   | <b>13</b> |
| 3.1      | Averaging Filter . . . . .                 | 13        |
| 3.2      | Gaussian Filter . . . . .                  | 14        |
| 3.3      | Median Filter . . . . .                    | 15        |
| 3.4      | Sobel Operator . . . . .                   | 16        |
| 3.5      | Laplacian Filter . . . . .                 | 17        |
| 3.6      | Laplacian Image Enhancement . . . . .      | 18        |
| <b>4</b> | <b>Image Magnification</b>                 | <b>20</b> |
| 4.1      | Magnification by Replication . . . . .     | 20        |
| <b>5</b> | <b>Summary Tables</b>                      | <b>21</b> |
| 5.1      | Quick Reference: Transformations . . . . . | 21        |
| <b>6</b> | <b>Practice Problems</b>                   | <b>21</b> |
| 6.1      | Problem 1: Contrast Stretching . . . . .   | 21        |
| 6.2      | Problem 2: Gamma Correction . . . . .      | 21        |
| 6.3      | Problem 3: Sobel Edge Detection . . . . .  | 21        |

# 1 Basic Gray Level Transformations

## 1.1 Contrast Stretching

### Definition

The process of expanding a narrow range of input intensities to fill the entire output range, thereby improving the contrast of the image.

#### Contrast Stretching Formula:

$$s = \frac{s_{\max} - s_{\min}}{r_{\max} - r_{\min}}(r - r_{\min}) + s_{\min}$$

where:

- $r_{\min}, r_{\max}$  = input range boundaries
- $s_{\min}, s_{\max}$  = output range boundaries
- $r$  = input intensity
- $s$  = output intensity

#### Practical Example:

An image has intensities limited to the range  $[50, 200]$ . We want to stretch this to the full  $[0, 255]$  range.

**Given Parameters:**

$$\begin{aligned} r_{\min} &= 50, & r_{\max} &= 200 \\ s_{\min} &= 0, & s_{\max} &= 255 \end{aligned}$$

**For pixel with intensity  $r = 100$ :**

$$\begin{aligned} s &= \frac{255 - 0}{200 - 50}(100 - 50) + 0 \\ &= \frac{255}{150}(50) \\ &= 1.7 \times 50 = 85 \end{aligned}$$

**For pixel with intensity  $r = 150$ :**

$$\begin{aligned} s &= \frac{255}{150}(150 - 50) + 0 \\ &= 1.7 \times 100 = 170 \end{aligned}$$

For pixel with intensity  $r = 175$ :

$$\begin{aligned} s &= \frac{255}{150}(175 - 50) + 0 \\ &= 1.7 \times 125 = 212.5 \end{aligned}$$

Results Table:

| Input (r) | Output (s) |
|-----------|------------|
| 50        | 0          |
| 100       | 85         |
| 150       | 170        |
| 200       | 255        |

**Observation:** The narrow input range  $[50, 200]$  has been expanded to the full output range  $[0, 255]$ , significantly improving image contrast.

## 1.2 Digital Negative

### Definition

Inverts the pixel intensities by reversing the intensity scale. Dark areas become bright and vice versa.

Digital Negative Formula:

$$s = L - 1 - r$$

where:

- $L$  = number of gray levels (typically 256 for 8-bit images)
- $r$  = input intensity
- $s$  = output intensity

**Practical Example:**

For an 8-bit image,  $L = 256$ , so the formula becomes  $s = 255 - r$ .

$$\text{For } r = 50 : \quad s = 255 - 50 = 205$$

$$\text{For } r = 100 : \quad s = 255 - 100 = 155$$

$$\text{For } r = 150 : \quad s = 255 - 150 = 105$$

$$\text{For } r = 200 : \quad s = 255 - 200 = 55$$

$$\text{For } r = 255 : \quad s = 255 - 255 = 0$$

**Results Table:**

| Input (r) | Output (s) | Description            |
|-----------|------------|------------------------|
| 0         | 255        | Black becomes White    |
| 50        | 205        | Dark becomes Bright    |
| 100       | 155        | Darker becomes Lighter |
| 150       | 105        | Medium becomes Medium  |
| 200       | 55         | Bright becomes Dark    |
| 255       | 0          | White becomes Black    |

**Visual Effect:** Complete intensity inversion, useful for enhancing details in dark medical images (e.g., mammograms).

### 1.3 Thresholding

#### Definition

Converts a grayscale image to a binary image by assigning pixels to two intensity levels based on a threshold value.

#### Thresholding Formula:

$$s = \begin{cases} 0 & \text{if } r < T \\ L - 1 & \text{if } r \geq T \end{cases}$$

where:

- $T$  = threshold value
- $r$  = input intensity
- $L$  = number of gray levels
- $s$  = output intensity (binary: 0 or 255)

#### Practical Example:

For an 8-bit image with threshold  $T = 128$ :

$$\begin{aligned} r = 50 < 128 &\Rightarrow s = 0 \quad (\text{Black}) \\ r = 100 < 128 &\Rightarrow s = 0 \quad (\text{Black}) \\ r = 128 \geq 128 &\Rightarrow s = 255 \quad (\text{White}) \\ r = 150 \geq 128 &\Rightarrow s = 255 \quad (\text{White}) \\ r = 200 \geq 128 &\Rightarrow s = 255 \quad (\text{White}) \end{aligned}$$

**Sample 3×3 Image Application:**  
**Original Grayscale Image:**

$$\begin{bmatrix} 100 & 120 & 140 \\ 110 & 200 & 150 \\ 90 & 180 & 160 \end{bmatrix}$$

**Binary Output (T=128):**

$$\begin{bmatrix} 0 & 0 & 255 \\ 0 & 255 & 255 \\ 0 & 255 & 255 \end{bmatrix}$$

## 1.4 Log Transformation

### Definition

Compresses the dynamic range of an image by applying a logarithmic transformation. Expands dark pixel values and compresses bright values.

**Log Transformation Formula:**

$$s = c \cdot \log(1 + r)$$

**where:**

- $c$  = scaling constant (determines output range)
- $r$  = input intensity ( $r \geq 0$ )
- $s$  = output intensity

**Practical Example:**

Let  $c = 50$  (scaling constant).

$$r = 0 : \quad s = 50 \cdot \log(1) = 50 \cdot 0 = 0$$

$$r = 10 : \quad s = 50 \cdot \log(11) = 50 \cdot 1.041 = 52.05$$

$$r = 100 : \quad s = 50 \cdot \log(101) = 50 \cdot 2.004 = 100.2$$

$$r = 1000 : \quad s = 50 \cdot \log(1001) = 50 \cdot 3.001 = 150.05$$

$$r = 10000 : \quad s = 50 \cdot \log(10001) = 50 \cdot 4.000 = 200.0$$

**Results Table:**

| Input (r) | $\log(1+r)$ | Output (s) |
|-----------|-------------|------------|
| 0         | 0           | 0          |
| 10        | 1.041       | 52.05      |
| 100       | 2.004       | 100.2      |
| 1000      | 3.001       | 150.05     |
| 10000     | 4.000       | 200.0      |

**Key Insight:** The logarithm compresses large input values into a smaller output range. Useful for visualizing Fourier spectra where values can range from 0 to millions.

## 1.5 Power-Law (Gamma) Transformation

### Definition

A non-linear transformation controlled by a gamma parameter. Used primarily for display device calibration and image enhancement based on perception.

### Power-Law (Gamma) Transformation Formula:

$$s = c \cdot r^\gamma$$

### where:

- $c$  = scaling constant (usually 1)
- $r$  = normalized input intensity,  $r \in [0, 1]$
- $\gamma$  = gamma parameter
- $s$  = output intensity

### Effect of $\gamma$ :

- $\gamma < 1$ : Brightens the image (expands dark values)
- $\gamma = 1$ : Identity transformation (no change)
- $\gamma > 1$ : Darkens the image (compresses dark values)

**Practical Example with  $c = 1$ :****Case 1: Brightening with  $\gamma = 0.4$ :**

$$r = 0.2 : \quad s = (0.2)^{0.4} = 0.725$$

$$r = 0.5 : \quad s = (0.5)^{0.4} = 0.860$$

$$r = 0.8 : \quad s = (0.8)^{0.4} = 0.956$$

**Case 2: Darkening with  $\gamma = 2.0$ :**

$$r = 0.2 : \quad s = (0.2)^{2.0} = 0.04$$

$$r = 0.5 : \quad s = (0.5)^{2.0} = 0.25$$

$$r = 0.8 : \quad s = (0.8)^{2.0} = 0.64$$

**Comparison Table:**

| Normalized $r$ | $\gamma = 0.4$<br>(Brighten) | $\gamma = 1.0$<br>(Identity) | $\gamma = 2.0$<br>(Darken) |
|----------------|------------------------------|------------------------------|----------------------------|
| 0.2            | 0.725                        | 0.2                          | 0.04                       |
| 0.3            | 0.788                        | 0.3                          | 0.09                       |
| 0.5            | 0.860                        | 0.5                          | 0.25                       |
| 0.7            | 0.915                        | 0.7                          | 0.49                       |
| 0.8            | 0.956                        | 0.8                          | 0.64                       |

**Observation:** Values with  $\gamma < 1$  are pushed upward (brightening), while values with  $\gamma > 1$  are pushed downward (darkening).



## 2 Histogram Processing

### 2.1 Histogram Equalization

#### Definition

A technique that transforms an image to have an approximately uniform histogram, automatically enhancing the contrast.

#### Histogram Equalization Formula:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) = (L - 1) \sum_{j=0}^k \frac{n_j}{M \times N}$$

#### where:

- $s_k$  = output intensity level
- $r_k$  = input intensity level
- $L$  = number of gray levels (256 for 8-bit)
- $p_r(r_j)$  = normalized histogram (probability)
- $M \times N$  = total pixels

#### Algorithm:

1. Compute histogram  $h(r_k)$  of input image
2. Compute normalized histogram:  $p(r_k) = \frac{h(r_k)}{M \times N}$
3. Compute CDF:  $\text{CDF}(k) = \sum_{j=0}^k p(r_j)$
4. Transform:  $s_k = \text{round}[(L - 1) \times \text{CDF}(k)]$
5. Map each pixel with intensity  $r_k$  to  $s_k$

**Complete Histogram Equalization Example:**

Consider a  $4 \times 4$  image with 8 gray levels (0-7):

**Original Image:**

$$\begin{bmatrix} 0 & 1 & 1 & 2 \\ 1 & 2 & 2 & 3 \\ 2 & 3 & 3 & 4 \\ 3 & 4 & 4 & 5 \end{bmatrix}$$

**Step 1: Count Pixels (Histogram)**

| Intensity Level | $n_k$ | Count     |
|-----------------|-------|-----------|
| 0               | 1     | 1 pixel   |
| 1               | 3     | 3 pixels  |
| 2               | 4     | 4 pixels  |
| 3               | 4     | 4 pixels  |
| 4               | 3     | 3 pixels  |
| 5               | 1     | 1 pixel   |
| <b>Total</b>    |       | 16 pixels |

**Step 2: Normalize Histogram**

$$p(r_k) = \frac{n_k}{16}$$

| Level      | $n_k$ | $p(r_k)$ | Probability |
|------------|-------|----------|-------------|
| 0          | 1     | 1/16     | 0.0625      |
| 1          | 3     | 3/16     | 0.1875      |
| 2          | 4     | 4/16     | 0.2500      |
| 3          | 4     | 4/16     | 0.2500      |
| 4          | 3     | 3/16     | 0.1875      |
| 5          | 1     | 1/16     | 0.0625      |
| <b>Sum</b> |       |          | 1.0000      |

**Step 3: Compute Cumulative Distribution Function (CDF)**

$$\text{CDF}(k) = \sum_{j=0}^k p(r_j)$$

| Level | CDF Calculation | CDF Value |
|-------|-----------------|-----------|
| 0     | 0.0625          | 0.0625    |
| 1     | 0.0625 + 0.1875 | 0.2500    |
| 2     | 0.2500 + 0.2500 | 0.5000    |
| 3     | 0.5000 + 0.2500 | 0.7500    |
| 4     | 0.7500 + 0.1875 | 0.9375    |
| 5     | 0.9375 + 0.0625 | 1.0000    |

**Step 4: Transform to Output Levels**

With  $L = 8$ , multiply CDF by  $(L - 1) = 7$ :

$$s_k = \text{round}[7 \times \text{CDF}(k)]$$

| Level | CDF    | $7 \times \text{CDF}$ | Rounded | $s_k$ |
|-------|--------|-----------------------|---------|-------|
| 0     | 0.0625 | 0.4375                | 0.44    | 0     |
| 1     | 0.2500 | 1.7500                | 1.75    | 2     |
| 2     | 0.5000 | 3.5000                | 3.50    | 4     |
| 3     | 0.7500 | 5.2500                | 5.25    | 5     |
| 4     | 0.9375 | 6.5625                | 6.56    | 7     |
| 5     | 1.0000 | 7.0000                | 7.00    | 7     |

**Mapping Created:**

$$0 \rightarrow 0, \quad 1 \rightarrow 2, \quad 2 \rightarrow 4, \quad 3 \rightarrow 5, \quad 4 \rightarrow 7, \quad 5 \rightarrow 7$$

**Step 5: Apply Mapping to Image**

**Equalized Image:**

$$\begin{bmatrix} 0 & 2 & 2 & 4 \\ 2 & 4 & 4 & 5 \\ 4 & 5 & 5 & 7 \\ 5 & 7 & 7 & 7 \end{bmatrix}$$

**Results and Observations:**

- Original histogram was concentrated in range  $[0, 5]$
- After equalization, levels are spread to  $[0, 7]$
- Output histogram becomes more uniform
- Contrast is automatically enhanced
- No manual parameters required

## 2.2 Histogram Statistics

**Definition**

Statistical measures computed from the histogram to characterize image properties.

**Histogram Statistics Formulas:****Mean (Average Intensity):**

$$m = \sum_{k=0}^{L-1} r_k \cdot p(r_k)$$

**Variance (Contrast Measure):**

$$\sigma^2 = \sum_{k=0}^{L-1} (r_k - m)^2 \cdot p(r_k)$$

**Standard Deviation:**

$$\sigma = \sqrt{\sigma^2}$$

**Practical Example:**

Using the 4×4 image from the histogram equalization example above.

**Calculate Mean:**

$$\begin{aligned} m &= \sum_{k=0}^5 r_k \cdot p(r_k) \\ &= 0(0.0625) + 1(0.1875) + 2(0.2500) + 3(0.2500) \\ &\quad + 4(0.1875) + 5(0.0625) \\ &= 0 + 0.1875 + 0.5 + 0.75 + 0.75 + 0.3125 \\ &= 2.5 \end{aligned}$$

**Calculate Variance:**

$$\begin{aligned} \sigma^2 &= \sum_{k=0}^5 (r_k - 2.5)^2 \cdot p(r_k) \\ &= (0 - 2.5)^2(0.0625) + (1 - 2.5)^2(0.1875) \\ &\quad + (2 - 2.5)^2(0.2500) + (3 - 2.5)^2(0.2500) \\ &\quad + (4 - 2.5)^2(0.1875) + (5 - 2.5)^2(0.0625) \\ &= 6.25(0.0625) + 2.25(0.1875) + 0.25(0.2500) \\ &\quad + 0.25(0.2500) + 2.25(0.1875) + 6.25(0.0625) \\ &= 0.3906 + 0.4219 + 0.0625 + 0.0625 + 0.4219 + 0.3906 \\ &= 1.75 \end{aligned}$$

**Calculate Standard Deviation:**

$$\sigma = \sqrt{1.75} \approx 1.32$$

**Interpretation:**

- **Mean brightness:** 2.5 out of 7 (on grayscale)
- **Variance:** 1.75 (measures spread/contrast)
- **Standard deviation:** 1.32 (typical deviation from mean)
- Higher variance indicates better contrast
- Used in adaptive image processing algorithms

### 3 Spatial Filtering

#### 3.1 Averaging Filter

##### Definition

A low-pass filter that replaces each pixel with the average of its neighborhood.

##### 3×3 Averaging Filter Kernel:

$$w = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

##### General Convolution Formula:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot f(x + s, y + t)$$

##### Practical Example:

Apply 3×3 averaging filter to a 5 × 5 image.

Original Image:

$$\begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 15 & 25 & 35 & 45 & 55 \\ 20 & 30 & 40 & 50 & 60 \\ 25 & 35 & 45 & 55 & 65 \\ 30 & 40 & 50 & 60 & 70 \end{bmatrix}$$

Apply filter at position (2,2):

Neighborhood (3×3):

$$\begin{bmatrix} 25 & 35 & 45 \\ 30 & 40 & 50 \\ 35 & 45 & 55 \end{bmatrix}$$

Calculation:

$$\begin{aligned} g(2, 2) &= \frac{1}{9}(25 + 35 + 45 + 30 + 40 + 50 + 35 + 45 + 55) \\ &= \frac{1}{9}(360) \\ &= 40 \end{aligned}$$

Apply filter at position (2,3):

Neighborhood (3×3):

$$\begin{bmatrix} 35 & 45 & 55 \\ 40 & 50 & 60 \\ 45 & 55 & 65 \end{bmatrix}$$

**Calculation:**

$$\begin{aligned} g(2, 3) &= \frac{1}{9}(35 + 45 + 55 + 40 + 50 + 60 + 45 + 55 + 65) \\ &= \frac{1}{9}(450) \\ &= 50 \end{aligned}$$

### 3.2 Gaussian Filter

**Definition**

A weighted averaging filter where weights follow a Gaussian distribution, giving more importance to the center pixel.

**3×3 Gaussian-like Kernel:**

$$w = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

**Practical Example:**

Using the same 3×3 neighborhood from the averaging filter example.

**Neighborhood:**

$$\begin{bmatrix} 25 & 35 & 45 \\ 30 & 40 & 50 \\ 35 & 45 & 55 \end{bmatrix}$$

**Calculation:**

$$\begin{aligned} g(2, 2) &= \frac{1}{16}[1(25) + 2(35) + 1(45) + 2(30) + 4(40) + 2(50) \\ &\quad + 1(35) + 2(45) + 1(55)] \\ &= \frac{1}{16}[25 + 70 + 45 + 60 + 160 + 100 + 35 + 90 + 55] \\ &= \frac{1}{16}(640) \\ &= 40 \end{aligned}$$

**Advantages of Gaussian Filter:**

- Center pixel has highest weight (4/16)
- More natural blur compared to box filter
- Better edge preservation
- Reduces block artifacts
- Smoother transitions

### 3.3 Median Filter

**Definition**

A non-linear filter that replaces each pixel with the median value of its neighborhood. Excellent for removing salt-and-pepper noise.

**Practical Example:**

Denoise a  $3 \times 3$  neighborhood with salt-and-pepper noise.

**Neighborhood with Noise:**

$$\begin{bmatrix} 10 & 15 & 20 \\ 12 & 255 & 18 \\ 14 & 16 & 22 \end{bmatrix}$$

Note: The value 255 is noise (white spot) in a region that should be dark.

**Step 1: Extract Values**

$$\{10, 15, 20, 12, 255, 18, 14, 16, 22\}$$

**Step 2: Sort**

$$\{10, 12, 14, 15, 16, 18, 20, 22, 255\}$$

**Step 3: Find Median**

With 9 values, the median is the 5th value:

$$\text{median} = 16$$

**Result:** Replace 255 with 16 (noise removed!)



**Key Properties:**

- Excellent for salt-and-pepper noise
- Preserves edges better than averaging
- Non-linear (cannot be expressed as convolution)
- Best choice for impulse noise removal

### 3.4 Sobel Operator

**Definition**

A  $3 \times 3$  gradient operator with smoothing, providing better edge detection than Roberts with more noise suppression.

**Sobel Kernels:**

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

**Gradient Magnitude:**

$$|f| = \sqrt{G_x^2 + G_y^2}$$

**Practical Example:**

Apply Sobel operator to a  $3 \times 3$  neighborhood.

**Neighborhood:**

$$\begin{bmatrix} 10 & 20 & 30 \\ 15 & 25 & 35 \\ 20 & 30 & 40 \end{bmatrix}$$

**Calculate  $G_x$ :**

$$\begin{aligned} G_x &= (-1)(10) + 0(20) + (+1)(30) \\ &\quad + (-2)(15) + 0(25) + (+2)(35) \\ &\quad + (-1)(20) + 0(30) + (+1)(40) \\ &= -10 + 30 - 30 + 70 - 20 + 40 \\ &= 80 \end{aligned}$$

**Calculate  $G_y$ :**

$$\begin{aligned}
 G_y &= (-1)(10) + (-2)(20) + (-1)(30) \\
 &\quad + 0(15) + 0(25) + 0(35) \\
 &\quad + (+1)(20) + (+2)(30) + (+1)(40) \\
 &= -10 - 40 - 30 + 20 + 60 + 40 \\
 &= 40
 \end{aligned}$$

**Gradient Magnitude:**

$$\begin{aligned}
 |f| &= \sqrt{80^2 + 40^2} \\
 &= \sqrt{6400 + 1600} \\
 &= \sqrt{8000} \\
 &\approx 89.44
 \end{aligned}$$

**Edge Direction:**

$$\theta = \tan^{-1} \left( \frac{40}{80} \right) = \tan^{-1}(0.5) \approx 26.57$$

### 3.5 Laplacian Filter

#### Definition

A second-order derivative operator that detects edges by finding zero-crossings. Produces thinner edges than first-derivative operators.

**Laplacian Mathematical Definition:**

$$\Delta^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

**Basic Laplacian Kernel (4-neighbors):**

$$\Delta^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

**Practical Example:**

Apply basic Laplacian to a  $3 \times 3$  neighborhood.

**Neighborhood 1 (Low contrast region):**

$$\begin{bmatrix} 10 & 20 & 30 \\ 15 & 25 & 35 \\ 20 & 30 & 40 \end{bmatrix}$$

**Apply Laplacian:**

$$\begin{aligned} {}^2f &= 0(10) + 1(20) + 0(30) \\ &\quad + 1(15) + (-4)(25) + 1(35) \\ &\quad + 0(20) + 1(30) + 0(40) \\ &= 20 + 15 - 100 + 35 + 30 \\ &= 0 \end{aligned}$$

**Neighborhood 2 (High contrast center):**

$$\begin{bmatrix} 10 & 20 & 10 \\ 20 & 50 & 20 \\ 10 & 20 & 10 \end{bmatrix}$$

**Apply Laplacian:**

$$\begin{aligned} {}^2f &= 0(10) + 1(20) + 0(10) \\ &\quad + 1(20) + (-4)(50) + 1(20) \\ &\quad + 0(10) + 1(20) + 0(10) \\ &= 20 + 20 - 200 + 20 + 20 \\ &= -120 \end{aligned}$$

**Interpretation:** Large negative value indicates strong edge/feature at center.

### 3.6 Laplacian Image Enhancement

#### Definition

Combines original image with Laplacian to enhance edges and fine details.

#### Laplacian Enhancement Formula:

$$g(x, y) = f(x, y) + c \cdot {}^2f(x, y)$$

where  $c = -1$  or  $c = +1$  depending on kernel type.

**Practical Example:**

Using the high-contrast neighborhood from above.

**Given:**

- Original pixel:  $f(2, 2) = 50$
- Laplacian:  $\Delta f = -120$
- Coefficient:  $c = -1$

**Calculate:**

$$\begin{aligned} g(2, 2) &= 50 + (-1)(-120) \\ &= 50 + 120 \\ &= 170 \end{aligned}$$

**Result:** The center pixel intensity increased from 50 to 170, sharpening the edge.

## 4 Image Magnification

### 4.1 Magnification by Replication

#### Definition

Simplest magnification method where each pixel is replicated into a block of identical values.

#### 2× Magnification Example:

Original 2×2 Image:

$$\begin{bmatrix} 100 & 150 \\ 200 & 250 \end{bmatrix}$$

After 2× Magnification (4×4 Image):

$$\begin{bmatrix} 100 & 100 & 150 & 150 \\ 100 & 100 & 150 & 150 \\ 200 & 200 & 250 & 250 \\ 200 & 200 & 250 & 250 \end{bmatrix}$$

**Observation:** Each pixel becomes a  $2 \times 2$  block.

#### 3× Magnification Example:

Original 2×2 Image:

$$\begin{bmatrix} 100 & 150 \\ 200 & 250 \end{bmatrix}$$

After 3× Magnification (6×6 Image):

$$\begin{bmatrix} 100 & 100 & 100 & 150 & 150 & 150 \\ 100 & 100 & 100 & 150 & 150 & 150 \\ 100 & 100 & 100 & 150 & 150 & 150 \\ 200 & 200 & 200 & 250 & 250 & 250 \\ 200 & 200 & 200 & 250 & 250 & 250 \\ 200 & 200 & 200 & 250 & 250 & 250 \end{bmatrix}$$

#### Properties:

##### Advantages:

- Simplest and fastest method
- No arithmetic operations needed and Easy to implement

##### Disadvantages:

- Creates blocky, pixelated appearance
- Jagged edges (staircase artifacts) and Poor visual quality

## 5 Summary Tables

### 5.1 Quick Reference: Transformations

| Transformation   | Formula                                                                            | Application         |
|------------------|------------------------------------------------------------------------------------|---------------------|
| Contrast Stretch | $s = \frac{s_{max}-s_{min}}{r_{max}-r_{min}}(r - r_{min}) + s_{min}$               | Low-contrast        |
| Digital Negative | $s = L - 1 - r$                                                                    | Medical imaging     |
| Log              | $s = c \log(1 + r)$                                                                | Fourier spectrum    |
| Power-law        | $s = cr^\gamma$                                                                    | Display calibration |
| Histogram Eq.    | $s_k = (L - 1) \sum_{j=0}^k p(r_j)$                                                | Auto enhancement    |
| Median Filter    | Select middle value                                                                | Noise removal       |
| Sobel            | $ f  = \sqrt{G_x^2 + G_y^2}$                                                       | Edge detection      |
| Laplacian        | $\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$ | Fine details        |

## 6 Practice Problems

### 6.1 Problem 1: Contrast Stretching

Find the output for  $r = 80$  when stretching from  $[30, 220]$  to  $[0, 255]$ .

**Solution:**

$$s = \frac{255 - 0}{220 - 30}(80 - 30) + 0 = \frac{255}{190}(50) \approx 67.1$$

### 6.2 Problem 2: Gamma Correction

Apply  $s = r^{0.5}$  to  $r = [0.25, 0.64]$ .

**Solution:**

$$s(0.25) = (0.25)^{0.5} = 0.5$$

$$s(0.64) = (0.64)^{0.5} = 0.8$$

### 6.3 Problem 3: Sobel Edge Detection

Given neighborhood:

$$\begin{bmatrix} 5 & 10 & 15 \\ 10 & 20 & 30 \\ 15 & 30 & 45 \end{bmatrix}$$

**Solution:**

$$G_x = 80, \quad G_y = 80$$

$$|f| = \sqrt{80^2 + 80^2} \approx 113.14$$